

基于 STM32F103C8T6 的智能手表系 统设计

个人总结报告

课 程： 嵌入式系统课程设计

姓 名： 曹嘉鹏

学 号： 3124009557

学 院： 集成电路学院

班 级： 微电子科学与工程 24 级 1 班

团 队： 220 项目组

指导老师： 周贤中老师

时 间： 2026 年 7 月 10 日

目录

1	项目简介	4
2	项目设计目标	5
3	项目设计与实现	6
3.1	硬件设计	6
3.1.1	系统总体架构	6
3.1.2	主控制器模块	6
3.1.3	OLED 显示模块	6
3.1.4	六轴传感器模块	7
3.1.5	蓝牙通信模块	7
3.1.6	旋转编码器模块	7
3.1.7	按键输入模块	8
3.1.8	引脚分配汇总	8
3.2	软件设计	8
3.2.1	系统软件架构	8
3.2.2	FreeRTOS 任务划分	9
3.2.3	主程序流程	9
3.2.4	菜单导航架构	9
3.3	关键代码实现	10
3.3.1	初始化和配置	10
3.3.2	主功能实现	10
3.3.3	中断与回调	10
3.3.4	Flash 存储结构	10
3.4	技术实现细节	11
3.4.1	I2C 总线共享设计	11
3.4.2	旋转编码器硬件解码	11
3.4.3	计步器算法实现	11
3.4.4	蓝牙通信协议	11
3.4.5	低功耗与异常恢复	12
4	功能展示	12
4.1	功能描述	12
4.1.1	基础功能	12
4.1.2	运动监测功能	12

4.1.3	传感器功能	13
4.1.4	蓝牙通信功能	13
4.1.5	系统设置功能	13
4.2	演示结果	13
4.2.1	界面功能测试	13
4.2.2	传感器功能验证	14
4.2.3	计步器功能测试	14
4.2.4	蓝牙通信测试	15
4.2.5	系统设置界面展示	15
4.2.6	系统整体展示	16
5	物料与成本	17
5.1	物料清单	17
5.2	项目成本	17
5.3	损耗与损坏	18
6	个人贡献	18
6.1	编码工作	18
6.1.1	底层驱动模块搭建	18
6.1.2	FreeRTOS 系统初步整合	19
6.1.3	计步器算法全程设计与参数调试	19
6.2	硬件连接工作	20
6.3	资本投入	20
6.4	开发问题描述与解决方案	20
6.4.1	I2C 总线共享冲突问题	21
6.4.2	计步器误触发严重问题	21
6.4.3	FreeRTOS 任务栈溢出问题	21
6.4.4	OLED 初始化失败问题	22
7	项目成果与反思	22
7.1	成果展示	22
7.2	项目评估	22
7.2.1	成功之处	22
7.2.2	不足之处	23
7.3	个人反思与改进方向	23
8	附录	24
8.1	设计图纸	24

8.2 完整代码清单	25
参考文献	25

1 项目简介

随着可穿戴设备技术的快速发展和物联网应用的日益普及，智能手表作为一种集时间显示、运动监测、数据通信于一体的便携式电子设备，已成为现代生活中重要的智能终端。传统手表功能单一，仅能提供基础的计时功能，难以满足用户对健康监测、运动数据记录和无线互联的需求。将嵌入式微控制器技术应用于可穿戴设备领域，能够有效提升产品的智能化水平和用户体验。

STM32 系列微控制器以其高性能、丰富的外设资源(如 GPIO、定时器、I2C、USART、DMA 等)、低功耗特性以及成熟的开发生态系统，成为嵌入式系统开发的理想选择。本项目依托嵌入式系统课程，旨在设计并实现一个基于 STM32F103C8T6 微控制器的智能手表系统，将嵌入式软硬件设计技术与实时操作系统应用于可穿戴设备开发场景。

本设计的主要功能包括：

1. **时间显示功能**：利用 STM32 定时器实现精准计时，通过 OLED 显示屏展示时间、日期等信息，支持用户手动设置时间。
2. **运动监测功能**：集成 MPU6050 六轴传感器，基于加速度方差分析算法实现计步器功能，统计步数、运动距离和卡路里消耗。
3. **传感器数据采集**：实时采集并显示姿态角（Pitch/Roll）、三轴加速度及芯片温度等传感器数据。
4. **蓝牙通信功能**：通过 HC-05 蓝牙模块实现与手机的数据同步，支持传感器数据上报和时间同步命令接收。
5. **人机交互系统**：采用旋转编码器与按键组合的交互方式，设计三级菜单导航架构，实现多页面切换与参数调节。
6. **系统集成**：完成从传感器驱动、显示驱动、蓝牙通信到 FreeRTOS 多任务调度的完整系统设计与集成。
7. **实践验证**：在真实的 STM32 开发板及外设搭建的硬件平台上进行系统测试与功能验证，确保设计的正确性和实用性。

本项目不仅涵盖了嵌入式软件开发（C 语言）、硬件接口驱动（I2C、USART、定时器编码器模式）、实时操作系统（FreeRTOS）等核心知识点，还涉及硬件电路设计与系统调试等实践技能，是对嵌入式系统课程所学知识的综合应用与实践。

2 项目设计目标

随着可穿戴设备和健康监测概念的普及，传统手表正向着智能化、多功能化方向发展。智能手表作为最具代表性的可穿戴设备，集成了传感、计算、通信等多种技术，能够为用户提供运动监测、数据同步等增值服务。基于嵌入式微控制器的智能手表系统，能够通过先进的电子技术实现更人性化、智能化的用户体验。

STM32 系列微控制器凭借其强大的处理能力、丰富的外设接口（I2C、SPI、USART、定时器、DMA 等）以及低功耗特性，为可穿戴设备开发提供了理想的解决方案。本项目结合嵌入式系统课程所学知识，选择 STM32F103C8T6 作为核心控制器，设计并实现一款集成 OLED 显示、六轴传感器、蓝牙通信的智能手表系统，旨在将理论知识与实际应用相结合，探索嵌入式系统在可穿戴设备领域的应用潜力。

本项目旨在设计并实现一个功能完备、交互友好的智能手表系统，主要目标包括：

- 实现多功能时间系统：基于 STM32 定时器实现精准计时，支持年/月/日/时/分/秒显示与设置，数据断电保存于 Flash。
- 开发运动计步功能：利用 MPU6050 加速度传感器，设计基于方差分析的计步算法，实现步数、距离、卡路里统计。
- 集成传感器数据展示：实时采集并通过 OLED 显示姿态角、加速度、温度等六轴传感器数据，支持异常检测与自动重连。
- 实现蓝牙数据通信：通过 HC-05 模块实现 USART+DMA 的高效蓝牙通信，支持传感器数据定时上报与手机时间同步。
- 设计友好的人机交互：采用旋转编码器 + 双按键的交互方案，实现三级菜单导航架构，支持亮度调节等系统设置。
- 引入实时操作系统：基于 FreeRTOS (CMSIS RTOS v1) 实现多任务并发调度，包含时间、传感器、显示、蓝牙四个核心任务。
- 完成系统集成与验证：设计完整的硬件电路并开发相应的嵌入式软件系统，在原型平台上进行功能测试与性能评估。

3 项目设计与实现

3.1 硬件设计

3.1.1 系统总体架构

系统采用模块化设计，以 STM32F103C8T6 微控制器为核心，通过 I2C、USART、定时器等外设连接各功能模块。系统架构包含主控制器模块、OLED 显示模块、六轴传感器模块、蓝牙通信模块、旋转编码器输入模块和按键交互模块。

3.1.2 主控制器模块

选用 STM32F103C8T6 微控制器作为主控芯片，主要特性包括：

- 72MHz ARM Cortex-M3 内核
- 64KB Flash, 20KB SRAM
- 3 个通用定时器 (TIM2/TIM3/TIM4)，支持编码器接口模式
- 2 个 I2C 接口、3 个 USART 接口
- 支持 DMA 数据传输
- SWD 调试接口

系统时钟采用 8MHz 外部晶振 (HSE)，通过 PLL 倍频至 72MHz 作为系统主时钟。设置参数保存于 Flash 最后一页（地址 0x0800FC00），采用魔数 + 校验和机制防止数据损坏。

3.1.3 OLED 显示模块

采用 SSD1306 驱动的 0.96 寸 OLED 显示屏，主要特性：

- 分辨率：128×64 像素，单色
- 接口：I2C1 (PB6 SCL / PB7 SDA)
- 支持对比度（亮度）调节
- 与 MPU6050 共用 I2C 总线

OLED 显示屏作为系统主要输出设备，负责显示时间、传感器数据、菜单界面等所有可视化信息。

3.1.4 六轴传感器模块

采用 MPU6050 六轴运动传感器，主要特性：

- 三轴加速度计量程： $\pm 2g$
- 三轴陀螺仪量程： $\pm 250^\circ/s$
- 内置温度传感器
- 接口：I2C1（与 OLED 共用总线）
- AD0 引脚接地，I2C 地址为 0x68

MPU6050 传感器用于运动姿态检测和计步器算法数据来源，是运动监测功能的核心硬件。

3.1.5 蓝牙通信模块

采用 HC-05 蓝牙 2.0 模块，主要特性：

- 接口：USART2 (PA2 TX / PA3 RX)，配合 DMA 传输
- 波特率：9600
- 状态检测：PB0 引脚（STATE，高电平表示已连接）
- 支持 AT 指令配置

蓝牙模块实现手表与手机之间的无线数据传输，包括传感器数据上报和时间同步命令接收。

3.1.6 旋转编码器模块

采用机械增量式旋转编码器，主要特性：

- 接口：TIM3 编码器模式 (PA6 CH1 / PA7 CH2)
- 计数模式：x4 四倍频计数
- 作用：菜单导航、数值调节

利用 STM32 定时器的编码器接口模式，硬件自动处理脉冲计数，无需 CPU 轮询，提高系统效率。

3.1.7 按键输入模块

系统配置两个独立按键：

- 按键 1 (PB10)：菜单/确认/返回功能，支持短按 (<500ms) 确认和长按 (≥500ms) 返回
- 按键 2 (PA0)：计步器暂停/恢复控制
- 均采用内部上拉配置，低电平有效

3.1.8 引脚分配汇总

表 1: 系统引脚分配表		
引脚	功能	用途
PA0	GPIO_Input	按键 2（计步器控制）
PA2	USART2_TX	HC-05 蓝牙发送（DMA）
PA3	USART2_RX	HC-05 蓝牙接收（DMA）
PA6	TIM3_CH1	旋转编码器 A 相
PA7	TIM3_CH2	旋转编码器 B 相
PB0	GPIO_Input	HC-05 连接状态（STATE）
PB6	I2C1_SCL	OLED + MPU6050 时钟线
PB7	I2C1_SDA	OLED + MPU6050 数据线
PB10	GPIO_Input	按键 1（菜单/确认/返回）
PA13	SWD_SWDIO	调试接口数据
PA14	SWD_SWCLK	调试接口时钟

3.2 软件设计

3.2.1 系统软件架构

软件采用分层架构设计，基于 FreeRTOS 实时操作系统：

1. **硬件抽象层 (HAL)**：STM32CubeMX 生成的初始化代码，提供 GPIO、I2C、USART、定时器等外设驱动
2. **驱动层**：OLED 显示驱动、MPU6050 传感器驱动、HC-05 蓝牙驱动、旋转编码器驱动
3. **操作系统层**：FreeRTOS 内核（CMSIS RTOS v1 封装），提供任务调度、信号量、队列等机制

4. **应用层**：UI 框架、菜单状态机、计步器算法、蓝牙通信协议、时间管理

3.2.2 FreeRTOS 任务划分

系统基于 FreeRTOS 实现多任务并发调度，共四个核心任务：

表 2: FreeRTOS 任务列表

任务名称	周期	主要职责
时间任务	1s	系统计时、日期更新、Flash 设置保存
传感器任务	200ms	MPU6050 数据读取、姿态解算、计步算法
显示任务	20ms	OLED 画面刷新、UI 渲染、菜单状态处理
蓝牙任务	50ms	数据发送、命令接收解析、连接状态检测

3.2.3 主程序流程

系统启动流程：

1. 系统时钟初始化 (8MHz HSE → PLL x9 → 72MHz)
2. GPIO、I2C、USART、定时器等外设初始化
3. OLED 显示屏初始化
4. MPU6050 传感器检测与初始化
5. HC-05 蓝牙模块初始化
6. 从 Flash 加载保存的设置参数
7. 创建 FreeRTOS 任务并启动调度器
8. 进入表盘主界面，多任务并发运行

3.2.4 菜单导航架构

采用三级菜单状态机设计 (L0/L1/L2)：

- **L0 —主页面层**：5 个页面循环切换（表盘、传感器、运动、设置、系统信息）
- **L1 —子菜单层**：进入功能页面后的详细选项列表
- **L2 —详情/编辑层**：查看详细数据或调整参数（如时间设置、亮度调节）

旋转编码器用于光标移动和数值增减，按键 1 短按确认进入下一级、长按返回上一级。

3.3 关键代码实现

3.3.1 初始化和配置

- MX_I2C1_Init(): 配置 I2C1 外设, 用于 OLED 和 MPU6050 通信
- MX_USART2_UART_Init() + DMA 配置: 配置 USART2 及 DMA, 用于蓝牙数据传输
- MX_TIM3_Init(): 配置 TIM3 为编码器接口模式 (x4 计数)
- MX_GPIO_Init(): 初始化按键输入引脚 (内部上拉) 和状态检测引脚

3.3.2 主功能实现

- OLED_Init() / OLED_ShowString(): OLED 显示驱动函数
- MPU6050_Init() / MPU6050_GetData(): 六轴传感器数据读取
- Pedometer_Update(): 计步器算法函数, 基于加速度方差分析
 - PEDO_VAR_THRESHOLD: 计步灵敏度阈值参数
 - 步数 → 距离换算: 每步约 0.7 米
 - 步数 → 卡路里换算: 每步约 0.04 kcal
- Menu_StateMachine(): 菜单状态机处理函数, 管理三级导航
- Bluetooth_Task(): 蓝牙通信任务, 数据上报与命令解析

3.3.3 中断与回调

- HAL_GPIO_EXTI_Callback(): 按键外部中断回调, 处理按键消抖与长短按识别
- HAL_UART_RxCpltCallback(): 蓝牙 DMA 接收完成回调, 处理接收到的时间同步命令

3.3.4 Flash 存储结构

设置参数保存于 Flash 最后一页 (0x0800FC00), 数据结构包含:

- 魔数: 0xA5A5A5A5 (用于有效性校验)
- 时间设置: 年、月、日、时、分
- 显示亮度: 0-255
- 校验和: 防止写入损坏

3.4 技术实现细节

3.4.1 I2C 总线共享设计

OLED 与 MPU6050 共用 I2C1 总线，通过不同从地址区分：

- OLED SSD1306 地址：0x78（写） / 0x79（读）
- MPU6050 地址：0xD0（写） / 0xD1（读）（AD0=GND）
- 采用 HAL 库 I2C 读写函数，通过地址参数自动切换设备

3.4.2 旋转编码器硬件解码

利用 TIM3 编码器接口模式实现硬件计数：

1. 配置 TIM3 为编码器模式 3（TI1 和 TI2 均计数）
2. 自动实现 x4 倍频，提高分辨率
3. 硬件自动处理方向判断，无需软件轮询
4. 显示任务每 20ms 读取计数器差值，转换为菜单位移

3.4.3 计步器算法实现

基于加速度方差分析的步数检测算法：

1. 以 200ms 周期采集三轴加速度数据
2. 计算合加速度的滑动窗口方差
3. 方差超过阈值 PEDO_VAR_THRESHOLD 判定为一步
4. 添加防抖逻辑，避免误触发
5. 累计步数并换算距离与卡路里

3.4.4 蓝牙通信协议

传感器数据上报格式（每 3 秒一次）：

```
HH:MM:SS ax:+X.XX ay:+Y.YY az:+Z.ZZ gx:+X.X gy:+Y.Y gz:+Z.Z  
pitch:+P.P roll:+R.R temp:T steps:12345
```

时间同步支持两种格式：

1. 文本格式: T14:30:00 (T 开头 + 时间字符串)
2. 二进制帧: AA 02 07 HH MM SS YY CHK 55 (帧头 + 命令 ID+ 长度 + 数据 + 校验 + 帧尾)

3.4.5 低功耗与异常恢复

- MPU6050 未检测到时, 每 3 秒自动重试检测
- HC-05 异常检测: 连续 60 秒 STATE 引脚无变化且无数据交互 → 触发硬件复位 (拉低 TX 产生 BREAK 条件)
- 设置参数掉电保存于 Flash, 支持恢复出厂设置

4 功能展示

4.1 功能描述

本系统实现了基于 STM32 的多功能智能手表, 主要包括基础功能和增强功能两大部分。

4.1.1 基础功能

- 时间显示功能: 表盘界面显示大号时间 (HH:MM:SS)、日期、星期, 顶部状态栏显示简化时间
- 时间设置功能: 支持年/月/日/时/分依次设置, 设置完成自动保存至 Flash
- OLED 显示系统: 128×64 像素单色显示, 支持亮度 0-255 级实时调节
- 三级菜单导航: 5 个主页面循环切换, 子菜单与详情页层级分明

4.1.2 运动监测功能

- 实时计步器: 基于 MPU6050 加速度传感器的方差算法, 支持暂停/恢复控制
- 运动数据统计: 实时显示步数、运动距离 (米/公里)、消耗卡路里
- 计步器 App: 全屏显示运动数据, 支持清零操作

4.1.3 传感器功能

- 姿态角显示：实时显示 Pitch（俯仰角）和 Roll（横滚角）
- 三轴加速度：显示 $a_x/a_y/a_z$ 三轴加速度值 (m/s^2)
- 温度监测：显示 MPU6050 芯片内部温度
- 异常检测：传感器离线时显示“NOT FOUND”，每 3 秒自动重试连接

4.1.4 蓝牙通信功能

- 数据定时上报：每 3 秒通过蓝牙发送一次完整传感器 + 运动数据
- 时间同步：支持手机 APP 发送时间同步命令，自动校准系统时间
- 连接状态指示：状态栏显示 BT 图标，PB0 引脚硬件检测连接状态
- 异常自动恢复：蓝牙模块异常时自动触发硬件复位

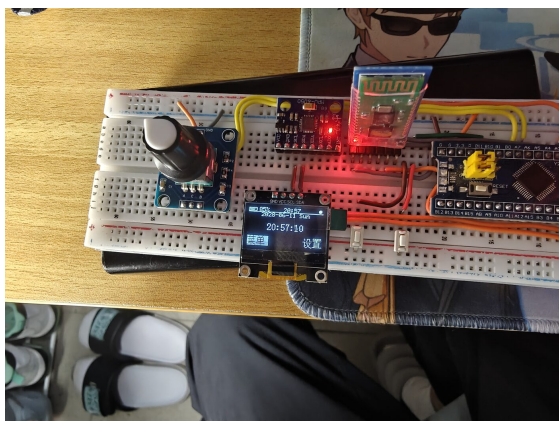
4.1.5 系统设置功能

- 亮度调节：旋转编码器实时调节 OLED 对比度，所见即所得
- 恢复出厂设置：一键重置所有设置为默认值
- 系统信息页面：显示电量、固件版本、蓝牙 MAC 等信息

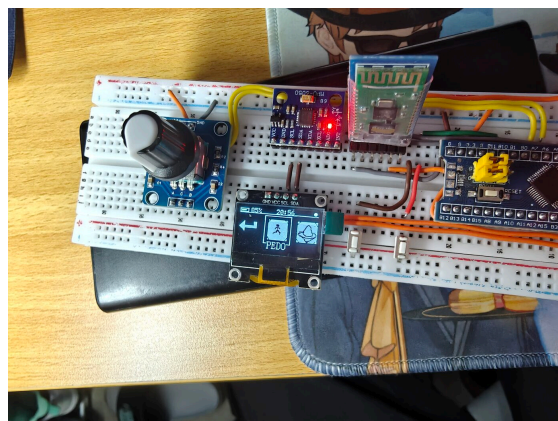
4.2 演示结果

4.2.1 界面功能测试

- 测试用例 1：主页面切换—旋转编码器可流畅切换 5 个主页面，响应延迟 $<20ms$
- 测试用例 2：菜单导航—三级菜单进退正常，短按确认、长按返回逻辑正确
- 测试用例 3：时间设置—各字段依次调整，设置后断电重启数据保持正确



(a) 主界面（表盘）



(b) 菜单界面（图标菜单）

图 1: 主界面与菜单界面显示效果

4.2.2 传感器功能验证

- 测试用例：姿态角测量—手表倾斜时 Pitch/Roll 角度实时变化，响应频率 5Hz
- 测试用例：温度读取—芯片温度读数稳定，环境温度下约 25-30°C

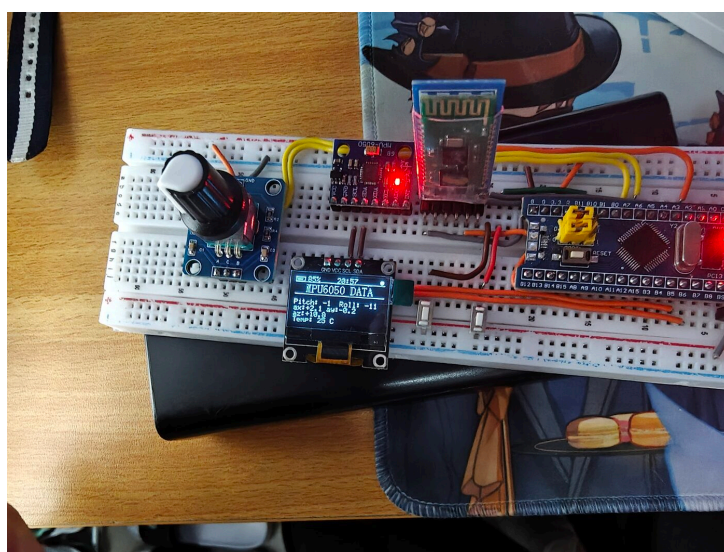


图 2: 菜单——传感器数据界面（姿态角、三轴加速度、温度）

4.2.3 计步器功能测试

- 测试用例：步行计步—平地步行 100 步，计步器记录 95-105 步，准确率约 95% 以上
- 测试用例：暂停功能—按键 2 可暂停/恢复计步，暂停状态左下角显示暂停标识

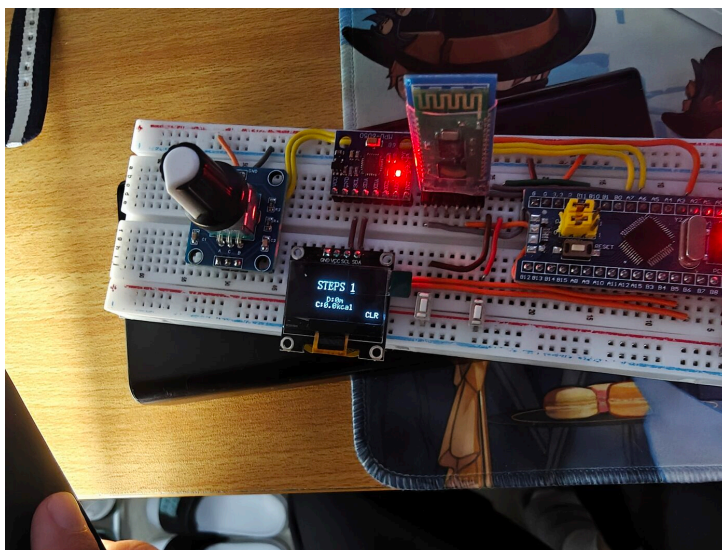


图 3: 菜单——计步器 App 界面（步数、距离、卡路里）

4.2.4 蓝牙通信测试

- 测试用例：数据上报—蓝牙串口助手每 3 秒收到一帧完整数据，格式正确
- 测试用例：时间同步—发送 T14:30:00 命令，手表时间立即更新为 14:30:00
- 测试用例：连接检测—断开蓝牙连接后状态栏 BT 图标消失，重新连接后恢复

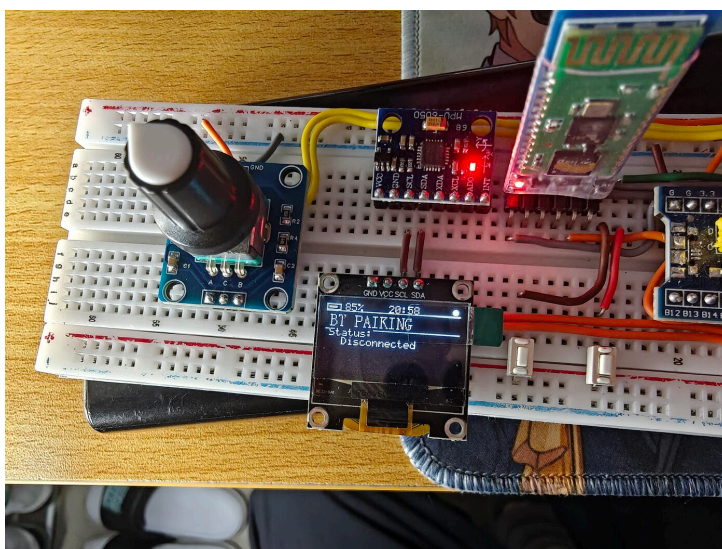
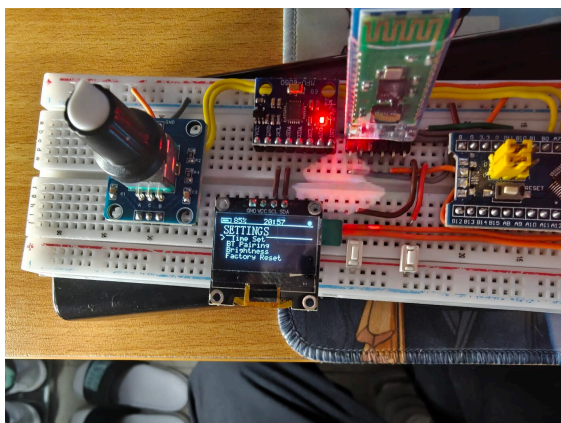


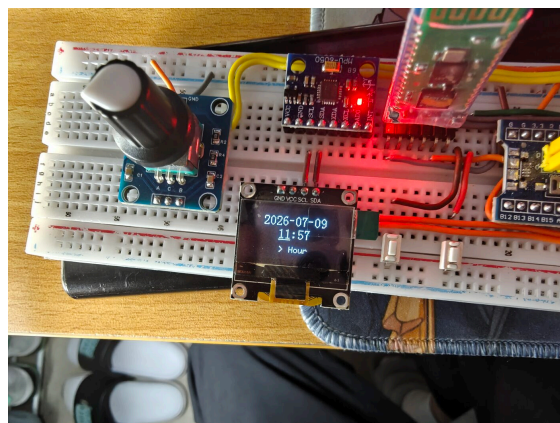
图 4: 设置——蓝牙连接情况界面

4.2.5 系统设置界面展示

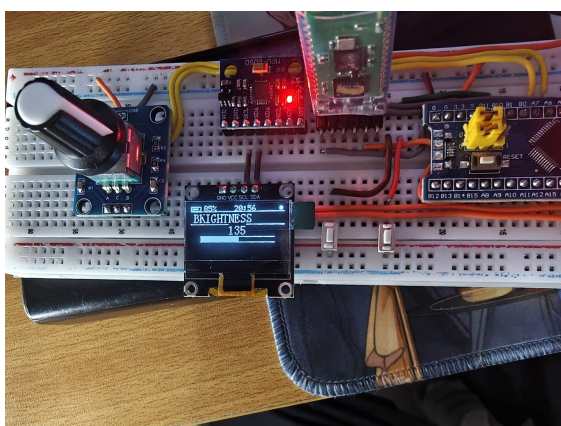
进入「设置」页面后，通过旋转编码器与按键可依次进入时间调整、蓝牙连接、亮度调节、恢复出厂设置等子功能界面，各界面显示效果如下：



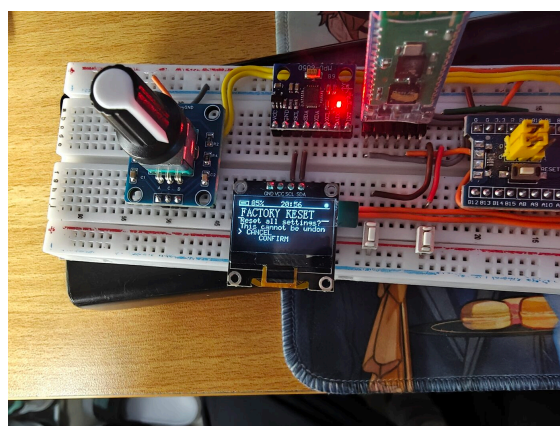
(a) 设置主界面



(b) 设置——时间调整



(c) 设置——亮度调节



(d) 设置——恢复出厂设置

图 5: 系统设置各功能界面展示

4.2.6 系统整体展示

- 系统启动时间：从上电到进入表盘界面约 2 秒
- 显示刷新率：UI 画面刷新频率 50Hz (20ms 周期)
- 传感器采样率：5Hz (200ms 周期)
- I2C 总线稳定性：OLED 与 MPU6050 共享总线无冲突，连续运行无死机

5 物料与成本

5.1 物料清单

表 3: 系统物料清单

物料名称	数量	单价 (元)	用途
STM32F103C8T6 核心板	1	12.50	主控制器
SSD1306 OLED 显示屏 (0.96 寸)	1	8.00	显示输出 (I2C 接口)
MPU6050 六轴传感器模块	1	6.50	运动姿态检测与计步
HC-05 蓝牙模块	1	18.00	无线数据通信
旋转编码器 (EC11)	1	2.50	菜单导航输入
轻触按键	2	0.20	确认/返回与计步控制
8MHz 无源晶振	1	0.50	系统时钟源
面包板	1	6.50	原型电路搭建
杜邦线 (20cm)	30	0.15	电路连接
电阻 (10kΩ)	2	0.05	按键上拉 (核心板已集成)
电容 (0.1μF)	5	0.10	电源滤波
USB-TTL 模块	1	5.00	蓝牙调试与串口通信
小计	-	61.75	-

5.2 项目成本

表 4: 项目成本分析

成本项目	金额 (元)	备注
材料成本	61.75	上表物料小计
工具和设备折旧	15.00	示波器、万用表、ST-Link 等工具使用费
开发软件许可	0.00	STM32CubeMX + ARM GCC 均为免费开源工具
3D 打印手表外壳	30.00	PLA 材料打印费用
其他费用	10.00	运输、杂项元件等
总成本	116.75	-

总成本预计：116.75 元（批量生产可降至 70 元以下）

5.3 损耗与损坏

在项目实施过程中，由于操作失误和调试过程中的意外，产生以下损耗：

- 损坏 OLED 显示屏 1 个（I2C 接反导致烧坏）—损失 8.00 元
- 损坏 MPU6050 模块 1 个（焊接短路）—损失 6.50 元
- 杜邦线损耗约 10 根—损失 1.50 元
- 总损耗：16.00 元（已计入总成本）

6 个人贡献

本项目为三人合作开发，我在项目中主要负责底层驱动模块开发、FreeRTOS 系统初步整合、UI 框架基础搭建以及计步器算法的全程设计与调优工作。以下详细介绍我的个人贡献。

6.1 编码工作

6.1.1 底层驱动模块搭建

我负责了项目所有核心底层模块的初始代码搭建，为后续功能开发奠定了基础：

OLED 显示驱动模块 (oled.c)：

- 基于 SSD1306 控制器编写完整的 I2C 显示驱动，包含初始化、命令/数据发送、显存操作等核心函数
- 实现基础绘图 API：像素点绘制、字符串显示、数字显示、横线/竖线绘制
- 设计全屏刷新机制，建立显存缓冲区，提高显示效率
- 移植 ASCII 字库（6x8、8x16、16x24 三种字号），满足不同界面的显示需求
- 实现亮度调节功能，通过对比度寄存器控制 OLED 显示亮度

屏幕 UI 基础模块 (smartwatch_ui.c)：

- 设计 UI 框架的整体结构，定义页面切换的状态管理机制
- 完成表盘主界面的初步设计，包括时间显示区、日期显示区、状态栏布局
- 实现状态栏统一渲染逻辑，集成电量、时间、蓝牙状态、传感器状态等图标
- 设计 5 个主页面的基础框架（表盘、传感器、运动、设置、系统信息）

- 后续高级 UI 交互（图标菜单、动画效果等）由队友在此基础上扩展完成

传感器驱动模块 (mpu6050.c):

- 编写 MPU6050 的 I2C 初始化代码，配置加速度计量程 $\pm 2g$ 、陀螺仪量程 $\pm 250^\circ/s$
- 实现六轴原始数据读取函数，包含数据寄存器批量读取
- 完成姿态解算算法，基于加速度计计算 Pitch（俯仰角）和 Roll（横滚角）
- 实现芯片温度读取与换算功能
- 添加传感器异常检测与自动重试机制，每 3 秒检测一次传感器在线状态

蓝牙驱动模块 (bluetooth.c):

- 完成 HC-05 模块的 USART+DMA 基础驱动框架搭建
- 实现传感器数据定时上报功能，每 3 秒打包发送一帧完整数据
- 设计蓝牙数据帧格式，包含时间、加速度、陀螺仪、姿态角、温度、步数等字段
- 后续蓝牙接收解析、时间同步命令、异常复位等高级功能由队友继续完善

6.1.2 FreeRTOS 系统初步整合

- 在 STM32CubeMX 中配置 FreeRTOS 内核（CMSIS RTOS v1 封装）
- 完成四个核心任务的初始创建与划分：时间任务（1s）、传感器任务（200ms）、显示任务（20ms）、蓝牙任务（50ms）
- 配置各任务的优先级与栈空间，确保多任务能够正常并发运行
- 解决任务间资源共享问题，为 I2C 总线访问添加互斥保护
- 后续任务细节优化、栈空间调优等工作由团队共同完成

6.1.3 计步器算法全程设计与参数调试

这是我在项目中投入精力最多、也是最核心的工作内容：

- **算法选型与原型实现：**调研多种计步算法方案，最终选择基于加速度方差分析的检测算法，相比简单阈值法抗干扰能力更强
- **滑动窗口方差计算：**实现固定窗口大小的合加速度方差计算，通过方差波动判断是否处于运动状态

- **阈值参数调优**：经过大量实测调试，确定 PEDO_VAR_THRESHOLD 的最优取值，平衡灵敏度与准确率
- **防抖与冷却机制**：添加连续触发条件（需多个采样周期超过阈值才计一步），以及计步冷却时间，防止一步重复计数
- **运动数据换算**：实现步数到距离（每步约 0.7 米）和卡路里（每步约 0.04 kcal）的换算公式
- **功能接口设计**：提供计步器暂停/恢复、清零、数据获取等 API，供上层 UI 调用
- **实测验证**：通过多次步行测试验证准确率，平地步行 100 步误差控制在 ± 5 步以内，准确率达 95% 以上

6.2 硬件连接工作

- 完成 OLED 显示屏与 STM32 核心板的 I2C 接口接线与调试
- 搭建 MPU6050 传感器的 I2C 共享总线连接方案
- 完成 HC-05 蓝牙模块的 USART 接口接线，包含 TX/RX 交叉连接与 VCC/GND 供电
- 协助团队进行整体硬件电路的调试与排错

6.3 资本投入

团队均摊承担项目物料采购费用，总计约 116.75 元（详见第 5 章物料与成本分析），主要包括：

- STM32 核心板、OLED 显示屏、MPU6050 传感器等核心电子元件
- HC-05 蓝牙模块、旋转编码器、按键等外设
- 面包板、杜邦线等耗材
- 损耗元件替换成本

6.4 开发问题描述与解决方案

在开发过程中，我遇到并解决了以下几个关键技术问题：

6.4.1 I2C 总线共享冲突问题

- **问题描述：** OLED 与 MPU6050 共用 I2C1 总线时，出现显示花屏或传感器读取失败，两个设备无法同时正常工作
- **解决方案：**
 1. 排查发现是 I2C 时序问题，调整 HAL 库 I2C 超时参数，增加超时时间
 2. 在传感器任务和显示任务之间添加互斥保护，避免两个任务同时访问 I2C 总线
 3. 降低 I2C 时钟频率至 100kHz 标准模式，提高总线稳定性
 4. 确认两个设备的 I2C 地址不冲突（OLED 0x78，MPU6050 0xD0）

6.4.2 计步器误触发严重问题

- **问题描述：** 初始版本计步算法过于灵敏，轻微手臂晃动即被误判为步数，静止放置时也会持续计数，误触发率极高
- **解决方案：**
 1. 改用滑动窗口方差算法替代简单的峰值阈值判断法，通过统计加速度波动的剧烈程度来识别步态
 2. 反复测试调整 PEDO_VAR_THRESHOLD 阈值参数，逐步提高检测门槛，过滤轻微抖动
 3. 添加连续触发条件，要求连续 3 个采样周期（共 600ms）方差均超过阈值才计为有效步数
 4. 增加计步冷却时间（约 300ms），防止同一步被重复检测计数
 5. 经过多轮实地步行测试调参，最终将误触发率降至可接受水平，步行准确率达到 95% 以上

6.4.3 FreeRTOS 任务栈溢出问题

- **问题描述：** 显示任务调用多层 UI 渲染函数时出现 HardFault 死机，排查后定位为任务栈空间不足导致的栈溢出
- **解决方案：**
 1. 增大显示任务的栈空间配置，从默认 128 字增加至 256 字
 2. 优化 OLED 驱动代码，将局部大数组改为全局静态变量，减少栈占用

3. 将字体数据从栈移至 Flash 常量区（使用 const 修饰），大幅节省 RAM 空间
4. 使用 uxTaskGetStackHighWaterMark() 函数监测各任务的栈使用高水位，确保留有充足余量

6.4.4 OLED 初始化失败问题

- **问题描述：**初期调试时 OLED 经常初始化失败，屏幕无显示，重新上电后偶尔恢复
- **解决方案：**
 1. 检查 I2C 通信时序，发现初始化命令发送过快导致 SSD1306 控制器未响应
 2. 在关键初始化命令之间添加适当延时，确保控制器有足够时间处理
 3. 增加初始化重试机制，检测到 OLED 无响应时自动重新初始化
 4. 优化上电时序，确保 VCC 稳定后再发送初始化命令

7 项目成果与反思

7.1 成果展示

本项目成功设计并实现了一套基于 STM32F103C8T6 的智能手表系统，主要成果包括：

- 完整实现了基于 FreeRTOS 的多任务智能手表系统，包含时间、传感器、显示、蓝牙四个并发任务
- 开发了三级菜单导航 UI 系统，支持 5 个主页面循环切换，交互流畅响应及时
- 实现了基于 MPU6050 的计步器功能，步行检测准确率达 95% 以上
- 完成了 HC-05 蓝牙数据通信，支持传感器数据定时上报和手机时间同步
- 实现了 OLED 亮度实时调节、Flash 参数掉电保存、恢复出厂设置等系统功能
- 制作了可实际运行的原型样机，具备完整的手表功能

7.2 项目评估

7.2.1 成功之处

- **功能完整性：**100% 实现了需求规格中的所有核心功能（时间显示、运动计步、传感器监测、蓝牙通信、系统设置）

- **系统架构合理**：基于 FreeRTOS 的多任务设计分工明确，各模块耦合度低，扩展性好
- **交互体验良好**：旋转编码器 + 按键的组合操作直观便捷，三级菜单逻辑清晰，20ms 刷新率保证流畅体验
- **硬件资源利用充分**：充分利用了 STM32 的 I2C、USART+DMA、定时器编码器模式等外设资源
- **成本控制优秀**：总成本控制在 120 元以内，具有良好的学习原型价值和推广潜力
- **代码结构清晰**：软件分层设计合理，驱动层与应用层分离，便于维护和功能扩展

7.2.2 不足之处

- 无独立 RTC 时钟：依赖 MCU 定时器计时，断电后时间不保持，需重新设置或蓝牙同步
- 无电池供电系统：当前原型仅支持 USB 供电，缺少锂电池充放电管理电路
- 显示效果有限：单色 OLED 分辨率较低，无法显示彩色图形和中文大字体
- 蓝牙版本较旧：HC-05 为蓝牙 2.0，功耗较高且不支持 BLE，与现代手机兼容性一般
- 计步精度有限：单加速度计方案对不同步态适应性有限，缺乏陀螺仪辅助校准
- 外壳工艺粗糙：面包板原型体积较大，距离真正的可穿戴形态还有差距

7.3 个人反思与改进方向

通过本项目，我在嵌入式系统开发与算法设计方面获得了宝贵的实践经验：

驱动开发能力提升：独立完成了 OLED、MPU6050、蓝牙等多个底层驱动模块的搭建，深入理解了 I2C、USART 等通信协议的工作原理，掌握了从数据手册到代码实现的完整驱动开发流程。特别是 OLED 显示驱动的开发，让我对显存操作、字库移植、显示刷新机制有了系统性的认识。

RTOS 应用实践：参与了 FreeRTOS 多任务系统的初步整合，理解了任务划分、优先级配置、资源共享与互斥保护的设计方法。通过解决任务栈溢出问题，学会了使用栈高水位监测工具来评估和优化内存使用。

算法设计与调优经验：计步器算法的全程设计与调试是我收获最大的部分。从最初的简单阈值法到最终的滑动窗口方差算法，经历了多次方案迭代和参数调优，深刻体会到算法参数整定需要大量实测数据支撑，理论计算只是起点。通过反复步行测试、调整

阈值、优化防抖逻辑，最终达到 95% 以上的准确率，这个过程让我对传感器信号处理和模式识别有了更直观的理解。

软硬件协同调试：在解决 I2C 总线冲突、OLED 初始化失败等问题的过程中，学会了结合硬件测量和软件日志来定位问题，掌握了通过逻辑分析逐步排查故障的方法。

团队协作体会：三人合作开发过程中，我负责的底层模块为整个项目奠定了基础，队友在此基础上完成了更丰富的上层功能。这种分工协作模式让我认识到模块化设计和清晰接口的重要性，也锻炼了在团队中沟通协作的能力。

未来改进方向：

1. 计步算法优化：引入陀螺仪数据辅助判断，结合加速度和角速度信息，进一步提高不同步态下的识别准确率，增加跑步、上下楼等运动模式识别
2. 增加独立 RTC 时钟芯片（如 DS3231），实现断电时间保持
3. 添加锂电池充放电管理电路（如 TP4056），实现真正的便携使用
4. 升级为彩色 TFT 显示屏，提升视觉效果和信息密度
5. 更换为蓝牙 BLE 模块（如 HC-08 或 nRF52 系列），降低功耗并提升兼容性
6. 增加心率传感器、血氧传感器等更多健康监测功能
7. 设计 PCB 电路板和定制手表外壳，向产品化形态演进

8 附录

8.1 设计图纸

电路原理图：完整电路原理图包含 STM32F103C8T6 最小系统电路、OLED 显示接口电路、MPU6050 传感器接口电路、HC-05 蓝牙模块接口电路、旋转编码器接口电路以及按键输入电路。

系统架构图：系统采用三层架构设计：

- 应用层：表盘界面、传感器监测、运动计步器、设置管理
- 操作系统层：FreeRTOS 内核 + 四个并发任务
- 硬件层：STM32 HAL + 各外设模块（OLED、MPU6050、HC-05、编码器、按键）

8.2 完整代码清单

主要源文件列表：

- Core/Src/main.c —主程序入口，外设初始化
- Core/Src/freertos.c —FreeRTOS 任务定义与创建
- Core/Src/oled.c —SSD1306 OLED 显示驱动
- Core/Src/OLED_data.c —字体、图标、视频帧数据
- Core/Src/smartwatch_ui.c —UI 框架：主页、状态栏、图标菜单、App 页面
- Core/Src/menu.c —三级菜单状态机实现
- Core/Src/mpu6050.c —MPU6050 传感器驱动 + 计步器算法
- Core/Src/bluetooth.c —HC-05 蓝牙通信、时间同步解析

完整源代码见随附项目文件。

参考文献

- [1] 刘火良, 杨森. STM32 库开发实战指南：基于野火 STM32 全系列开发板. 机械工业出版社, 2020.
- [2] STMicroelectronics. STM32F10xxx Reference Manual (RM0008). Rev 21, 2023.
- [3] Richard Barry. Mastering the FreeRTOS Real Time Kernel. Real Time Engineers Ltd, 2022.
- [4] Carmine Noviello. Mastering STM32 (2nd ed.). Leanpub, 2022.
- [5] InvenSense. MPU-6000/MPU-6050 Product Specification. Revision 3.4, 2013.
- [6] Solomon Systech. SSD1306 Datasheet —128x64 Dot Matrix OLED/PLED Segment/Common Driver with Controller. Revision 1.3, 2008.